Mini proyecto Angular: Lista de tareas

Material de aprendizaje

Ruta de aprendizaje

- Crear un proyecto Angular con componentes standalone.
- Implementar un modelo de datos TypeScript para tareas.
- Construir un formulario interactivo para agregar y eliminar tareas.

APARTADO 01

Apartado 1: Creación del proyecto e implementación completa

Estructura de Directorios del Proyecto

A continuación se muestra el árbol de carpetas y archivos que componen la aplicación Angular standalone para la gestión de tareas.

```text
mi-app-tareas/
% % %  src/
%    % % %  app/
%    %    % % %  task.model.ts
%    %    % % %  app.component.ts
%    %    % % %  app.component.html
%    % % %  main.ts
%    % % %  index.html
% % %  package.json
```

Paso 1: Inicialización del proyecto

1. Ejecuta el comando en tu terminal para crear el proyecto: npx @angular/cli new mi-app-tareas --routing=false --style=css
2. Entra a la carpeta del proyecto creado: cd mi-app-tareas
3. Inicia el servidor de desarrollo local: ng serve
4. Abre tu navegador en la dirección http://localhost:4200 para comprobar que Angular compila correctamente.

Paso 2: Creación del modelo de datos

Crea el archivo task.model.ts para definir la estructura TypeScript de cada tarea.

```typescript
src/app/task.model.ts
export interface Task {
  title: string;
}
```

Comprobación: El modelo define correctamente una interfaz con la propiedad title de tipo string.

Paso 3: Implementación del componente principal

Configura app.component.ts utilizando componentes standalone, FormsModule y la lógica para añadir y eliminar tareas.

APARTADO 02

Apartado 1: Creación del proyecto e implementación completa (continuación)

TYPESCRIPT

src/app/app.component.ts

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Task } from './task.model';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  tasks: Task[] = [];
  newTaskTitle: string = '';

  addTask() {
    if (this.newTaskTitle.trim() !== '') {
      this.tasks.push({ title: this.newTaskTitle });
      this.newTaskTitle = '';
    }
  }

  deleteTask(index: number) {
    this.tasks.splice(index, 1);
  }
}
```

Comprobación: El componente compila sin errores e incluye los métodos addTask y deleteTask.

Paso 4: Creación de la plantilla HTML

Escribe el contenido completo de app.component.html con el formulario de entrada y la lista de elementos.

APARTADO 03

Apartado 1: Creación del proyecto e implementación completa (continuación)

HTML

src/app/app.component.html

```
<div style="max-width: 400px; margin: 40px auto; font-family: sans-serif;">
  <h2>Lista de Tareas</h2>

  <form (submit)="addTask(); $event.preventDefault()">
    <input
      type="text"
      [(ngModel)]="newTaskTitle"
      name="newTaskTitle"
      placeholder="Nueva tarea..."
      required
      style="padding: 8px; width: 70%; margin-right: 8px;"
    />
    <button type="submit" style="padding: 8px;">Agregar</button>
  </form>

  <ul style="padding-left: 20px; margin-top: 20px;">
    <li *ngFor="let task of tasks; let i = index" style="margin-bottom: 8px;">
      {{ task.title }}
      <button (click)="deleteTask(i)" style="margin-left: 10px;">Eliminar</button>
    </li>
  </ul>
</div>
```

Comprobación: La interfaz muestra el campo de texto, el botón de agregar y renderiza dinámicamente las tareas con opción de borrado.